

Complete AI Engineer Training: Python, NLP, Transformers, LLMs, LangChain, Hugging Face, APIs

Section 10: Python Module: Why Python?

Programming Explained in Few Minutes

Introduction to Programming

Programming begins with a simple observation: in everyday life, we encounter tasks that range from simple to complex. While some can be solved manually, more complex problems often require the assistance of a computer.

To solve a problem using a computer, two conditions must be met:

- The problem must be clearly defined
- The steps required to solve it must be known

However, even if these steps are written perfectly in plain English, a computer cannot understand them directly. Computers operate using **binary logic**, meaning they understand only:

- Zeros and ones
- Two states, similar to a light switch: ON and OFF

Source Code and Programs

To communicate with a computer, we must translate human logic into a format it can process.

- **Source code (human-readable code):**
 - A structured form of text written in a programming language
 - Interpreted or compiled into machine-level instructions (zeros and ones)
- **Program:**
 - A sequence of instructions that defines how to perform a computation

Definition of Programming:

Programming is the process of taking a task and expressing it in a programming language so that a computer can understand and execute it.

Programming vs Computer Science

A common misconception among beginners is confusing programming with computer science.

- **Computer Science:**
 - Studies what computers can do
 - Focuses on theory, computation, and systems
- **Programming:**

- The practical activity of instructing computers to perform tasks

These are related but fundamentally different disciplines.

Programming Languages and Their Purpose

There are thousands of programming languages, each designed for specific types of problems.

- Examples:
 - **PHP** → Web development
 - **C++** → Systems programming and device-level applications
 - **Python** and **R** → Data science and finance

An experienced programmer typically does not know all languages. Instead:

- They specialize in a few
- They develop deep expertise in those languages

Core Skills Required for Programming

Programming is not just about writing code—it is fundamentally about thinking.

1. Problem-Solving and Abstract Thinking

- Understand the problem completely
- Break it down into smaller, logical steps
- Translate those steps into computational instructions

Example Problem:

- Task: Add 10 to any number input by a user
- Logical reasoning:
 - Let input be **x**
 - Output should be **x + 10**

2. Mechanistic Thinking

- Computers execute only explicit instructions
- They do not infer intent or meaning
- Every step must be clearly defined

This requires:

- Precision
- Clarity
- Structured thinking

Commented [VP1]: to form an opinion or decide that something is true from the information you have

Writing Clear and Structured Code

Once the logic is defined, the next step is implementation.

- Code must be:
 - Clean
 - Readable
 - Well-structured

Poor coding practices include:

- Unclear variable names
- Overly complex logic
- Disorganized structure

In real-world scenarios:

- Programs often contain hundreds of lines of code
- Code is shared across teams
- Readability becomes critical for collaboration

Example: Breaking Down a Problem

Consider the earlier example of adding 10 to a number.

A programmer must think in terms of subtasks:

- Define a function
- Accept input x
- Compute $x + 10$
- Return the result

This structured breakdown is essential regardless of:

- The programming language used
- The complexity of the problem

Importance of Syntax and Code Literacy

To write effective programs, you must:

- Understand the syntax of a programming language
- Be able to read and interpret code
- Write instructions that the computer can execute unambiguously

Even when working with AI tools:

- Strong programming skills help you understand system behavior
- Enable debugging and troubleshooting
- Allow better integration of components

Developing Programming Skills

Programming ability improves through practice and structured learning.

- Key activities:
 - Solving programming challenges
 - Practicing problem decomposition
 - Writing and reviewing code

These activities strengthen:

- Problem-solving skills
- Mechanistic thinking
- Logical structuring

Final Perspective

Programming is the discipline of translating human thought into machine-executable instructions. It requires clarity, structure, and precision in thinking.

Mastering programming is not about memorizing syntax alone—it is about learning how to:

- Formulate problems
- Break them into logical steps
- Communicate those steps effectively to a machine

This structured way of thinking remains essential, even in an era increasingly influenced by artificial intelligence.

Why Python

Introduction to Python

Although it may be new to some of you, Python has been on the programming stage for over two decades.

There are two main reasons you should learn Python:

- It has several technical advantages compared to other programming languages.
- Its practical application covers several industries.

It is a powerful computational tool when solving complicated tasks in fields such as:

- Finance
- Econometrics
- Economics
- Data Science
- Machine Learning

Therefore, it is a perfect stepping stone for somebody who learns how to code and is determined to pursue a career as a data scientist.

Technical Definition of Python

Python is:

- Open source
- General purpose
- High level programming language

Let us break down each of these attributes.

Open Source

Open source means:

- It is free to use.
- It has a large and active scientific community.
- Users have access to the software's source code.
- The community contributes to continuous development and upgrades based on user needs.

This is one of the main reasons Python is cross-platform.

- Available on major operating systems:
 - Windows
 - Mac
 - Linux

Benefit:

Python can be quickly applied anywhere.

In comparison:

- Domain-specific languages like:
 - Matlab
 - SAS
- These are paid tools used for financial and econometric tasks.

This cost factor plays a role in a language's popularity.

General Purpose

Python is not limited to a single domain.

- It can be applied across a broad set of fields.
- Example:
 - Web development using the Django framework

Although web development may be beyond the scope here, it highlights:

- The wide range of applications
- Interoperability with other programming languages

This flexibility explains why many large organizations choose Python as their main programming language.

High Level Language

This is a more technical concept.

- Computers can directly execute only low-level (machine) languages.
- Programs written in high-level languages must first be translated into low-level code before execution.

This translation is handled by specialized software.

Implications:

- The translation process takes time.
- However, the benefits outweigh this cost.

Advantages of high-level languages:

- Low-level languages are difficult to:
 - Code
 - Understand
- High-level languages:
 - Use syntax closer to human logic
 - Are easier to learn and implement
 - Allow focus on problem-solving instead of decoding complex instructions

Summary of Technical Advantages

Python is often preferred because:

- It is free and constantly updated.
- It is applicable across multiple domains.
- It processes calculations efficiently.
- It has intuitive and readable syntax.
- It supports complex quantitative computations.

Practical Applicability and Big Data

Python's real strength is demonstrated through its practical use.

We are living in the era of big data.

- Disciplines generating large datasets:
 - Economics
 - Finance
 - Computer Science
 - Marketing

Definition of Big Data:

- Datasets with millions of observations

Traditional tools like Microsoft Excel become insufficient in such scenarios.

Limitations of Excel:

- Performance issues with large datasets
- Limited scalability

Need:

- A more powerful and flexible computational tool

Python fulfills this need by:

- Handling large datasets efficiently
- Providing flexibility across domains
- Enabling consistent approaches to data processing

Conclusion

Python's popularity is based on two main pillars:

1. **Ease of Learning**
 - Highly readable syntax
 - Clear and intuitive structure
2. **Power and Capability**
 - Executes complex computations
 - Suitable for advanced technical tasks

Despite being user-friendly, Python remains one of the most powerful programming languages preferred by specialists.

Section 11: Python Module: Setting Up the Environment

Jupyter – Introduction

Installing Python and Jupyter

For this course, you must install both Python and Jupyter on your computer.

- If you already have them installed, you can still proceed with this lecture.
- The lecture also introduces important concepts about how Jupyter works.

Why Python Is Not a Single Installable Application

A common question is:

Why isn't there a single software application called Python that:

- Installs easily
- Updates automatically
- Runs everything seamlessly

The answer lies in understanding what Python actually is.

- Python is a **programming language**, not a complete software application.
- It allows you to communicate with a computer.
- To execute Python code, you need additional software.

Role of Jupyter Notebook

To work with Python, you need an interface.

- **Jupyter Notebook (Jupyter)** is one such application.
- It is a **server-client application**.
- It allows you to write and execute code through a **web browser**.

Jupyter Architecture: Kernels and Clients



Jupyter operates using two main components:

1. Language Kernels

- Programs designed to:
 - Read
 - Execute code
- Each kernel supports a specific programming language:
 - Python
 - R
 - Julia
- Jupyter comes with:
 - A pre-installed Python kernel
- Additional kernels can be installed as needed

2. Clients (Interfaces)

- Interfaces where you write and interact with code
- Example:
 - Web browser

How It Works

- The **Jupyter server** connects:
 - A client (interface)
 - To a language kernel

In this course:

- Language: Python
- Client: Web browser (interactive shell)

Notebook Files (.ipynb)



Your work in Jupyter is stored in:

- **Notebook documents**
- File format: `.ipynb` (IPython Notebook)

These files contain:

- Code
- Text explanations

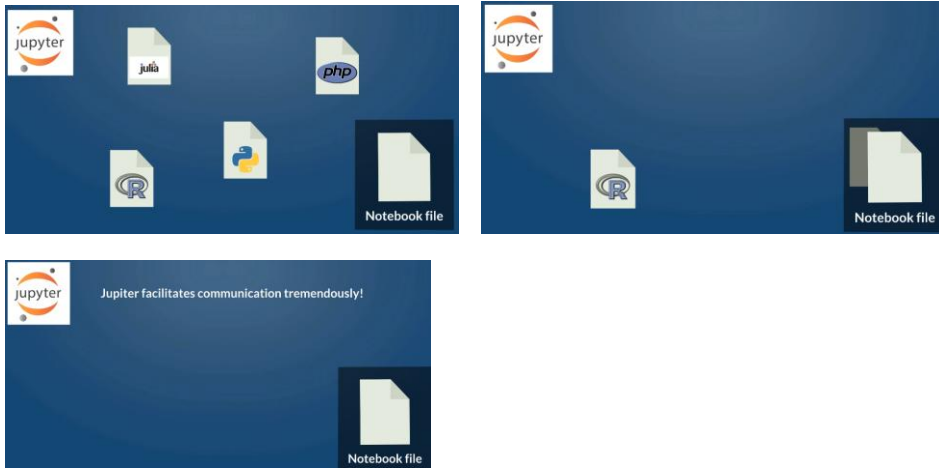
- Outputs

Why Jupyter Is Widely Used

Jupyter is used in large organizations such as:

- Google
- Microsoft
- IBM

Reason 1: Multi-language Support

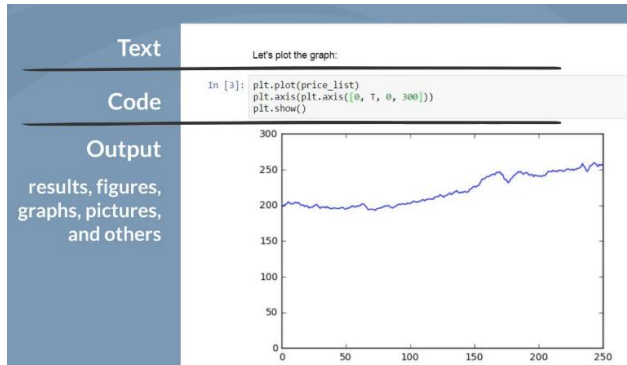


- Real-world tasks may require multiple languages:
 - Python
 - R
 - Julia
 - PHP
- Instead of switching tools:
 - Jupyter provides a unified interface
 - Each notebook connects to a chosen kernel

Benefit:

- Consistent workflow across different programming languages

Reason 2: Integrated Workflow



Unlike traditional text editors:

- Jupyter allows multiple content types in one file:
 - Plain text (explanations)
 - Code (e.g., Python)
 - Outputs:
 - Results
 - Graphs
 - Figures
 - Images

Benefit:

- No need to switch between tools
- Improves readability and understanding
- Ideal for:
 - Learning
 - Demonstrations
 - Documentation

Reason 3: Collaboration and Sharing

- Notebook files can be:
 - Stored locally
 - Hosted on remote servers

Benefit:

- Easy sharing across teams
- Improves collaboration in organizations

Why Jupyter Is Preferred

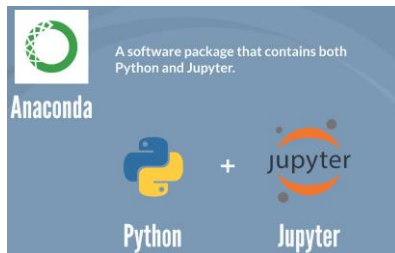
Jupyter simplifies the workflow by:

- Combining code, explanation, and output

- Supporting multiple programming languages
- Enabling easy sharing and collaboration

Because of these advantages, it is increasingly preferred over other software tools.

Next Step: Installing Anaconda



The recommended way to set up the environment is:

- Install **Anaconda**

Anaconda includes:

- Python programming language
- Jupyter Notebook application

This makes installation simple and efficient.

Summary

- Python is a programming language, not standalone software.
- Jupyter provides an interface to write and execute Python code.
- It uses:
 - Kernels (execution engines)
 - Clients (interfaces like browsers)
- Notebook files (.ipynb) integrate code, text, and output.
- Jupyter is widely used due to:
 - Flexibility
 - Multi-language support
 - Collaboration capabilities

It is the primary tool that will be used throughout the course.